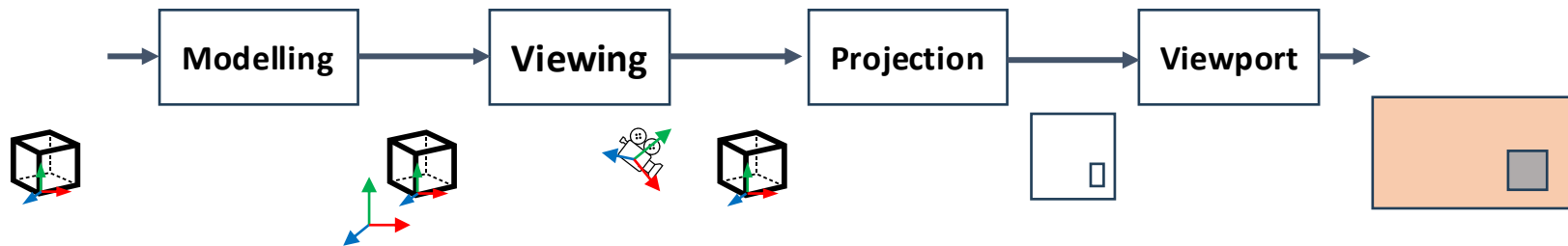


Computer Graphics Methods & Summary

P. J. Narayanan
Spring 2026

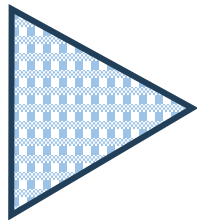


Rasterization Pipeline



We draw triangles mostly

- Project each vertex
- Assemble triangle
- Clip, Scan-convert to find pixels!



Pixels are filled at the end:

- Evaluate lighting at each 3D point
- At vertices and interpolate inside
- Interpolate normals, etc.



Image Generation by Computer

- Computer graphics generates images on the computer from a description of the scene.
- Goal of realism: Appearance as close to the real-world scene as possible.
- Computer Games require ~50–60 fps — speed is the priority.
- Animated Movies prioritise quality over speed.

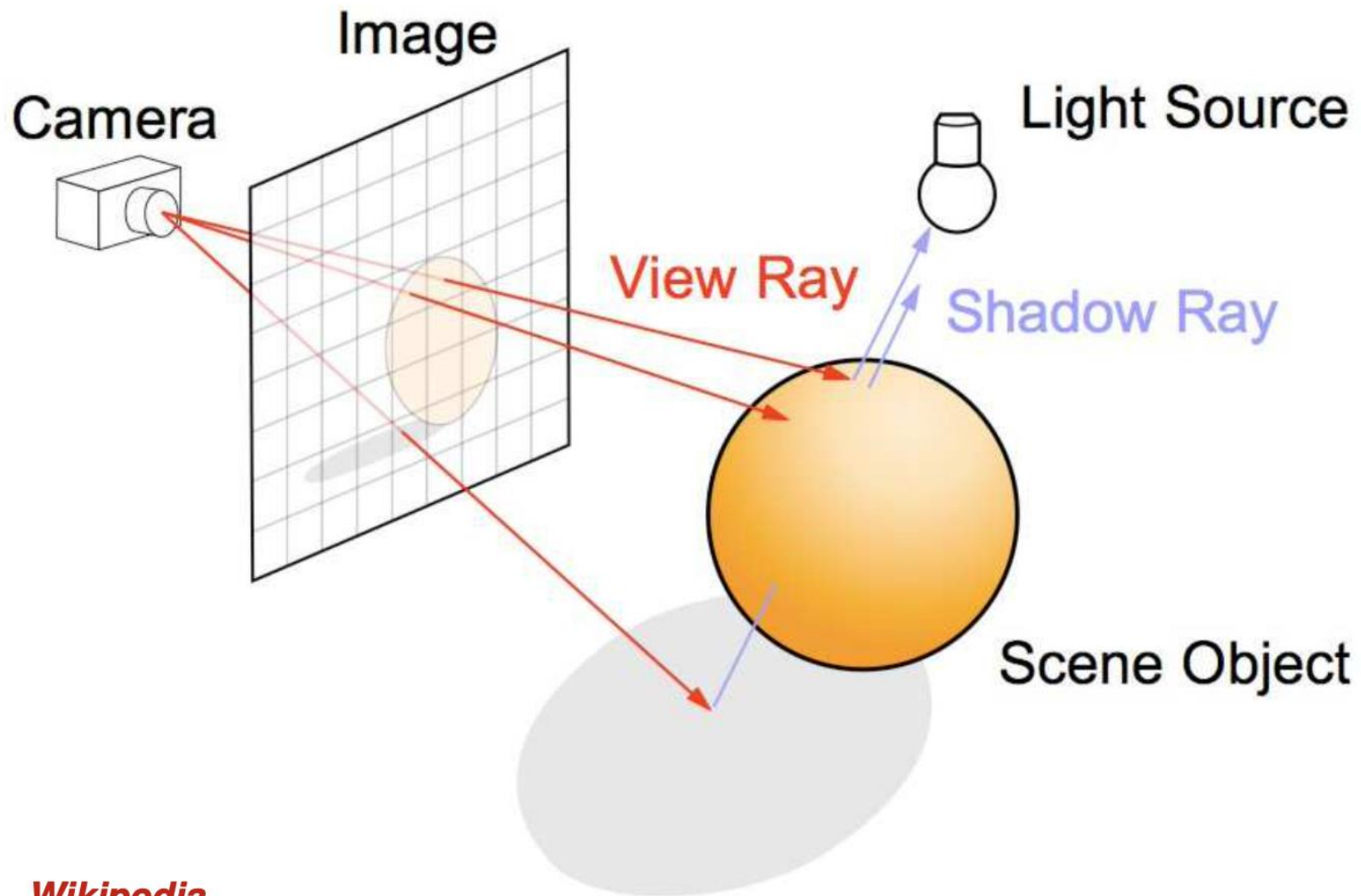




Looking at the Image

- An image has $M \times N$ pixels. Each pixel needs a colour.
- The colour at each pixel is what the ray from the camera through the pixel brings.
- There are infinitely many world points, but only MN pixels.
- Cast a ray from the camera centre through each pixel into the world.
- What does the ray hit? Colour the pixel accordingly!





Ray Tracing – Core Idea

- Send rays from CoP through each image pixel to the world and see what each finds

for each pixel in the image

Determine closest object in the direction of projector

Draw the pixel with appropriate colours

- Called **ray tracing** or **ray casting**.
- Equation of the ray is known. World objects are known. Need to intersect the ray with objects!



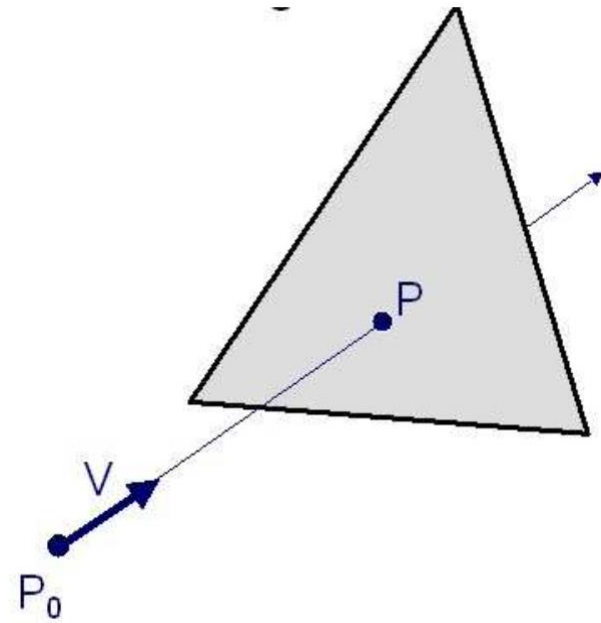
Ray Equation

- CoP is $P_o = (x_o, y_o, z_o)$ and the pixel point is $P_1 = (x_1, y_1, z_1)$.
- Ray:
$$P = P_o + t(P_1 - P_o) = (x_o + t \cdot \Delta x, y_o + t \cdot \Delta y, z_o + t \cdot \Delta z), \quad t > 0$$
- Negative t values are behind the CoP
 $t = 1$ is the pixel plane.
- Region of interest: $t > 1$, i.e. in front of the camera.
- Intersect with all objects; the closest object has the smallest positive t value.



Intersection with a Polygon

- To intersect the ray with a finite triangle, first intersect with the (infinite) plane.
- Plane equation: $Ax + By + Cz + D = 0$
- Intersection: $t = \frac{Ax_0 + By_0 + Cz_0 + D}{A \cdot \Delta x + B \cdot \Delta y + C \cdot \Delta z}$
- Is it inside the triangle? How?
- Efficient: **barycentric coordinates**



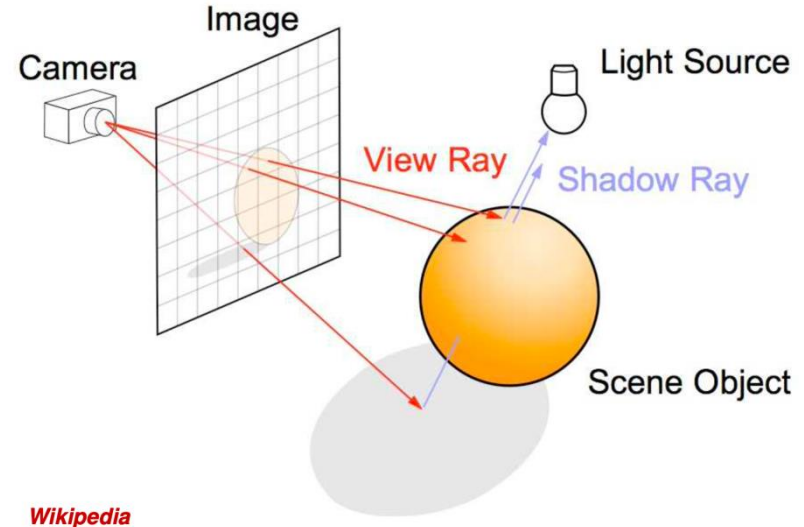
Scene with Multiple Primitives

- We now know how to intersect a ray with a **single** primitive.
- A typical scene contains many primitives.
- Naïve approach: loop over every primitive, keep the one with the minimum t .
- Inefficient: each ray must test against every primitive
- Acceleration structures needed for scalability



More needed!

- We also have light, shadow, reflections, refractions, etc.
- We want to include all of them
- **Recursive Ray Tracing!**



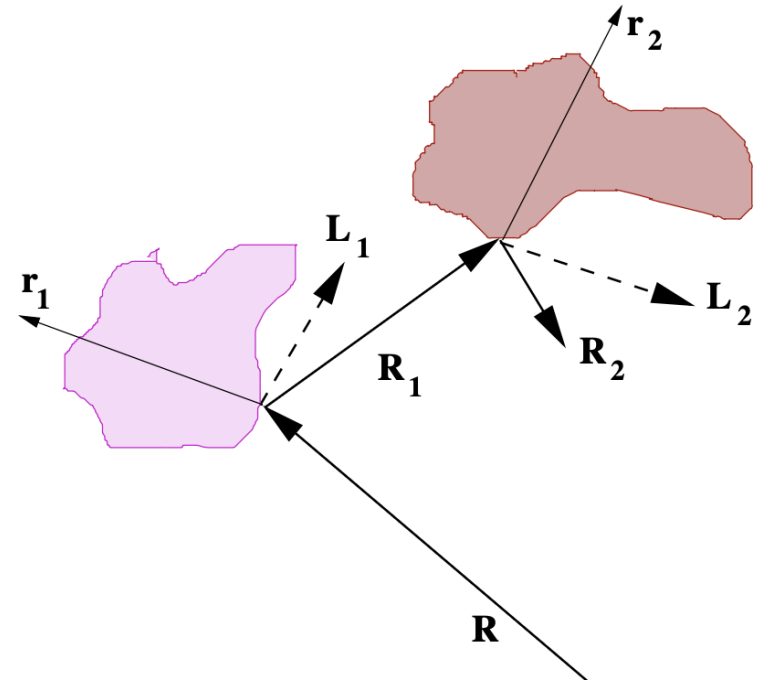
Recursive Ray Tracing

- When a *primary* ray hits an object, it spawns *secondary* rays
 - ❖ Reflection ray — reflected about the surface normal
 - ❖ Refraction (transmission) ray — bent as per Snell's law.
 - ❖ Shadow rays — cast toward each light source.
- ***Secondary rays*** apply the same principle recursively, accumulating colour/intensity.
- The final pixel colour is a combination of all individual contributions.



Recursive Ray Tracing

- New rays are spawned from each intersection point
- They are handled the same way
- Each returns a colour. Combine them at the primary, camera ray!!



RR Tracing: Algorithms

```
for each scan-line do
  for each pixel in scan-line do
    Determine the ray for the pixel
    pixelColour  $\leftarrow$  RTTrace(ray, depth=1)
```

```
RT Trace(ray, d)
  Find the closest object for the ray
  if object found:
    Compute  $\mathbf{nrml}$  at intrsct point
    return RTShade(obj, ray, intrsct,  $\mathbf{nrml}$ , d)
  else:
    return BackgroundColour
```

```
RTShade(obj, ray, pnt, n, d)
  clr = ambient term
  for each light L do:
    lRay = ray from pnt to light
    if lRay is blocked  $\rightarrow$  shadow
    else:  $\text{clr} += k_d * \text{diffuse from L}$ 

  if  $d \geq \text{maxDepth}$ : return clr

  if object is reflective:
    rRay = reflected ray;
     $\text{clr} += k_s * \text{RTTrace}(\text{rRay}, d+1)$ 

  if object is transparent:
    tRay = refracted ray;
     $\text{clr} += k_t * \text{RTTrace}(\text{tRay}, d+1)$ 
  return clr
```

When to Stop?

- Recursion never stops in the real world.
- In practice, the contribution of a deeper ray to the original pixel diminishes rapidly.
- Stop when further contribution is negligible.
- Simple rule: set a maximum depth limit MAX_DEPTH
- How do we speed it up? Use **acceleration structures**
 - ❖ Grid, Octree, BVH, KD-Tree, ...
 - ❖ Rich literature!



Monte Carlo Path Tracing

- Sample multiple points on each pixel and send rays out (uniformly within the pixel area).
- Evaluate and integrate the rendering equation along each path.
- At each intersection, spawn more rays sampled from the BRDF distribution.
- Uniform sampling for diffuse; other distributions for specular / glossy.
- Use Russian roulette to probabilistically terminate rays at each bounce.



What Is Path Tracing?

Monte Carlo Path Tracing (MCPT) is the mathematical simulation of light transport. It solves the Rendering Equation by tracing photons in reverse — from the camera back to the light source.

Photorealism

Naturally accounts for soft shadows, ambient occlusion, and color bleeding without artist-defined hacks.

Unbiased

Given enough samples, the algorithm converges to the mathematically correct solution — guaranteed.



Kajiya's Integral (1986)

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega} f_r(p, \omega_i, \omega_o) L_i(p, \omega_i) (\omega_i \cdot n) d\omega_i$$

L_o

Outgoing radiance — the final pixel color

L_e

Emitted light — if the surface is itself a light source

f_r

BSDF — how the material reflects or transmits light

L_i

Incoming radiance — light arriving from other scene points

Estimate Numerically

$$L_o \approx \frac{1}{N} \sum_{j=1}^N \frac{\text{contribution of sample } j}{\text{probability of picking that sample}}$$



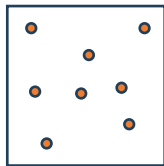
Monte Carlo Integration

The integral covers every direction in a hemisphere — impossible to solve analytically. Use statistical sampling instead.



Samples Per Pixel

Shoot N random rays through each pixel. More samples $\rightarrow$ less noise.



Averaging

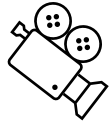
Final pixel color = average contribution of all N traced paths.



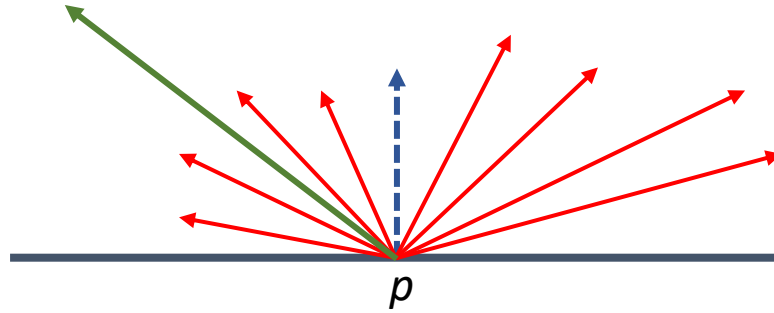
Convergence Rate

To halve noise, quadruple SPP. Noise reduces as $1/\sqrt{N}$.





$$f_r(p, \omega_i, \omega_o)$$



Incoming $L_i(p, \omega_i)$ is another integral for the point/ray recursively



BSDF: How Materials Interact With Light

BRDF — Reflective

Bidirectional Reflectance Distribution Function

Diffuse (Lambertian)

Scatters light equally in all directions. Matte surfaces like clay or concrete.

Specular

Perfect mirrors or glossy lobes modeled via microfacet theory (GGX, Beckmann).

BTDF — Transmissive

Bidirectional Transmittance Distribution Function

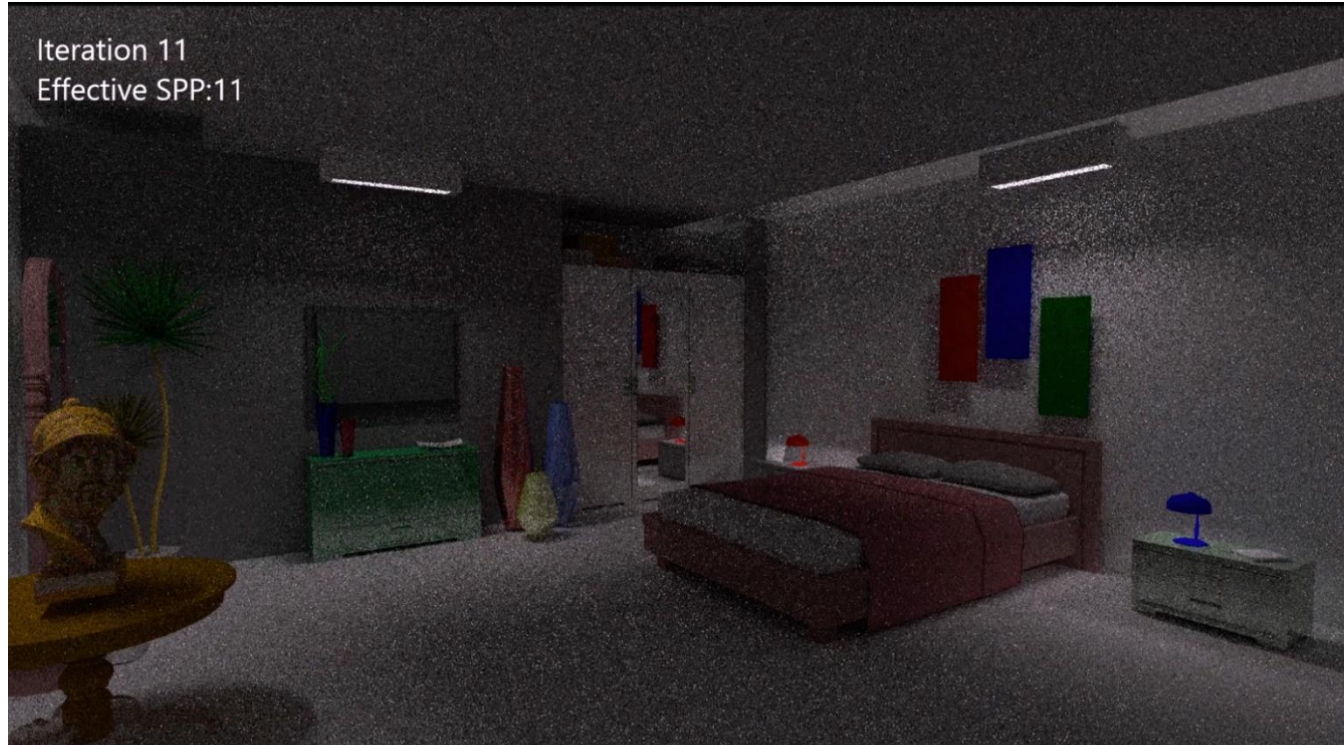
Refraction (Snell's Law)

Light bends as it crosses material boundaries. Used for glass and water.

Total Internal Reflection

Light trapped inside high-IOR mediums — responsible for sparkling gems.

Impact of Samples Per Pixel



[Link to the video](#)





[Link to the video](#)

Accelerating Convergence

Naïve path tracing is slow. Modern renderers use smart sampling strategies to find light faster and reach clean renders with fewer samples.

01

Importance Sampling

Instead of picking any random direction, bias samples toward directions where the BSDF is most likely to reflect significant energy. Fewer wasted rays.

02

Next Event Estimation (NEE)

At every bounce, explicitly cast a shadow ray directly toward light sources to solve for direct illumination. Dramatically reduces variance in scenes with small lights.

03

Russian Roulette

Probabilistically terminate low-energy paths without introducing bias. Keeps computation focused on paths that matter most.



Path Tracing

MCPT

Solves the rendering equation by reverse-tracing light paths

Monte Carlo

Statistical sampling trades exactness for scalability

BSDF

Material models (BRDF + BTDF) define how light scatters

Sampling

IS + NEE + Russian Roulette dramatically cut render times



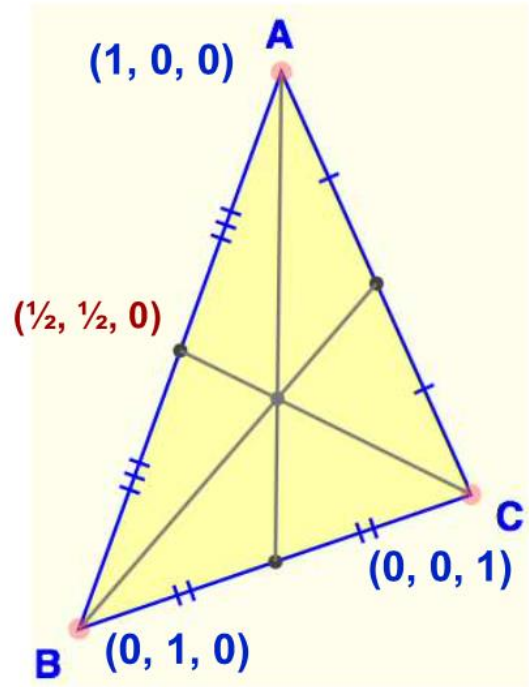
Ray Tracing: Summary

- Ray tracing produces realistic images by simulating light transport.
- Acceleration structure is critical to reduce the number of intersection tests.
- Path tracing (Monte Carlo ray tracing) is the standard method in production rendering today.
- Interesting mathematics: geometry, probability, physics — all combined.
- Hardware support (NVIDIA RTX) enables real-time ray tracing.



Barycentric Coordinates on Triangles

- Point $P = \alpha A + \beta B + \gamma C$, where α , β , γ are the **barycentric** coordinates with $\alpha + \beta + \gamma = 1$.
- Midpoints of sides:
midpoint of AB = $(\frac{1}{2}, \frac{1}{2}, 0)$.
- Centroid = $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$.
- Inside the triangle: $\alpha, \beta, \gamma \in [0, 1]$.
Outside if any coordinate < 0 or > 1 .



Barycentric Coordinates on Triangles

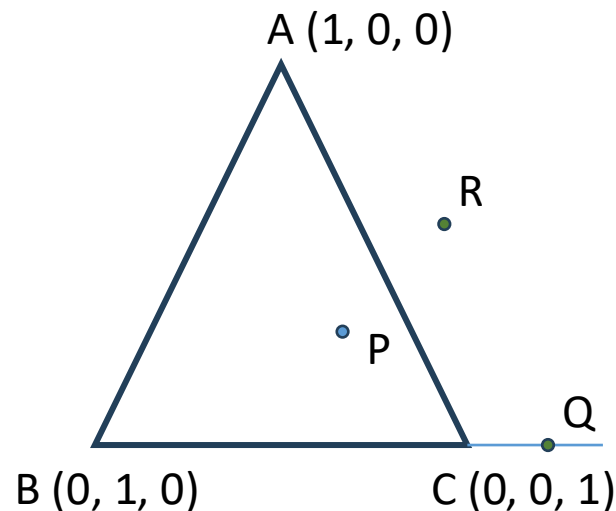
- Given vertices A, B, C and point P

- $P = \alpha A + \beta B + C(1 - \alpha - \beta)$

- $[\alpha, \beta]^T = \mathbf{T}^{-1}(P - C)$ where

$$\mathbf{T} = [A - C \quad B - C]$$

- Is $\mathbf{T}$ invertible? Why?
- Eliminate special cases



Final Words

- Computer Graphics is an exciting field
- A lot of progress in the past 60+ years
- Combination of Maths, Physics, Algorithms
- Quest is for ultra-realism today
- Speed is always a concern
- Bright future ahead still



The End!!

Thank You!

